

Final Project Report

Course 4- Intro to Machine Learning

Student Name: Muhammad Kamran Syed

Contents

Loan Default Prediction - Final Report.....2

 Introduction2

 Methods and Materials2

 Step-by-Step Process with Visualization and Comparison Chart:2

 Results21

 Discussion21

 Conclusion21

Loan Default Prediction - Final Report

Introduction

Loan default prediction is a critical problem in the financial sector, where lenders and financial institutions seek to minimize risk while optimizing lending decisions. With the increasing availability of financial data, machine learning techniques provide a powerful tool to assess credit risk and predict potential defaulters. Identifying high-risk borrowers enables institutions to implement appropriate risk mitigation strategies, thereby reducing financial losses and improving decision-making processes.

This study aims to explore various machine learning models to develop a predictive model for loan defaults. The primary objective of the study is to determine which model provides the most accurate and reliable predictions based on available financial data. Through this project, we investigate different feature selection techniques, data balancing strategies, and model optimization methods to enhance predictive performance. Our key research questions include:

- What are the most important financial indicators influencing loan defaults?
- Which machine learning model offers the best balance between accuracy and recall?
- How do data balancing techniques impact model performance?

Methods and Materials

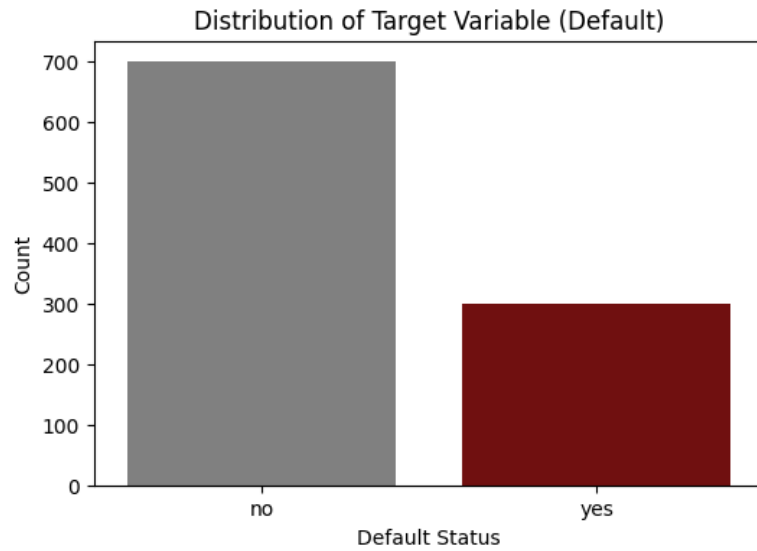
To ensure a systematic approach, the study was conducted using various machine learning techniques, starting with Exploratory Data Analysis (EDA). The dataset underwent preprocessing steps, including checking and handling duplicates and missing values, feature encoding, and normalization. Data visualization was employed to identify trends and relationships among financial attributes, such as loan amount, credit history, income level, and savings balance.

For modeling, we tested multiple machine learning algorithms, including Logistic Regression, Support Vector Machine (SVM), Decision Trees, Random Forest, and Gradient Boosting methods such as LightGBM and XGBoost. To address the issue of class imbalance, we implemented the Synthetic Minority Over-sampling Technique for Nominal and Continuous (SMOTENC) and weighted sampling. Hyperparameter tuning was conducted using GridSearchCV to optimize model performance. The models were evaluated using performance metrics such as Confusion Matrix, Classification Performance Report, Accuracy, Precision, Recall, F1-score, ROC-AUC, and Error Rate.

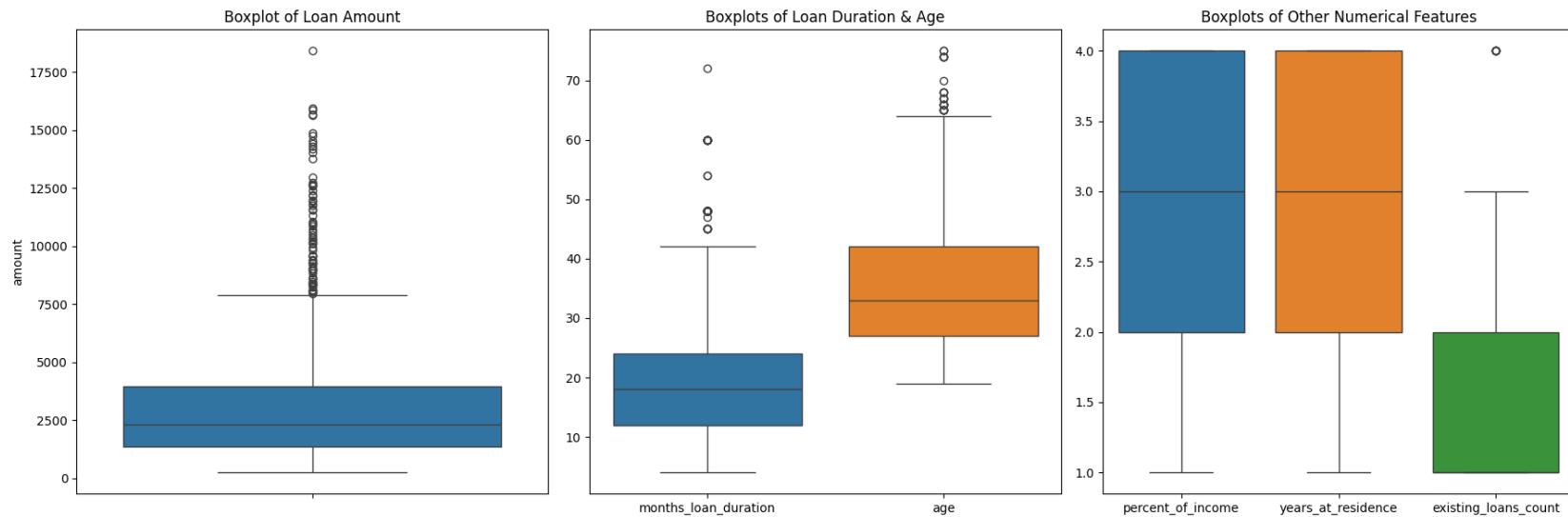
Step-by-Step Process with Visualization and Comparison Chart:

Data Preprocessing and Exploratory Data Analysis (EDA)

- Loaded the dataset from a CSV file and checked for data integrity.
- Performed an initial exploratory data analysis (EDA) to understand feature distributions and detect anomalies.



Feature Distribution for Target Variable (Default) was 70% No Default, 30% Default. We identified that the data is imbalanced.

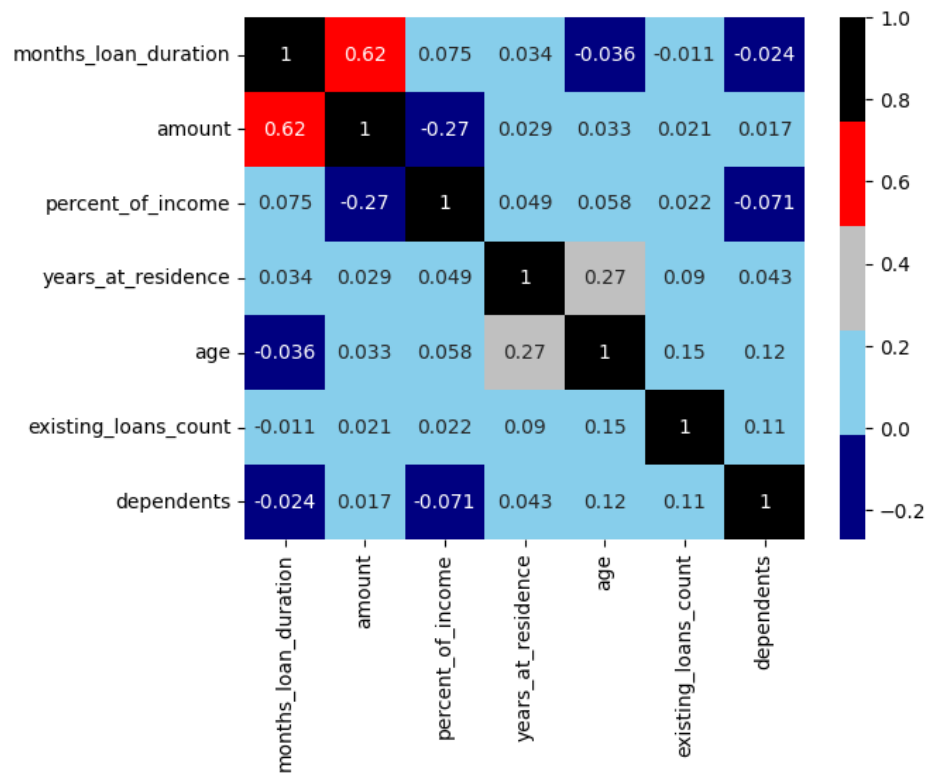


Check for outliers, however in loan default prediction, extreme values (e.g., very high loan amounts, very high or low balances) are genuine scenarios representing actual customers, not data errors or anomalies. Our data was also limited, hence, we didn't treat outliers.



Check the Distribution of Other Numerical Features and Find that:

- Loan Amount and Duration are positively correlated.
- Younger customers are more likely to default compared to older customers
- Customers paying a higher percentage of their income on loans appear to default more often
- Customers with multiple existing loans show a higher chance of default.



Look for any Correlation between features

- Amount and Duration are highly positively correlated.
 - There is a weak -ve correlation between Loan Amount and percentage of Income.
 - There is a weak +ve correlation between Amount and Phone.
 - There is a weak +ve correlation between Age and Years at Residence
- Checked for missing and duplicate values. No duplicates or missing values were present in our dataset.
 - Binary encoded Binary values columns i.e. Phone and Default.
 - Fix the Data sanitization issues by renaming car0 to car in the feature name Purpose.
 - Encode Purpose column using 1 hot encoding (pd.get_dummies()), since it represent nominal values.

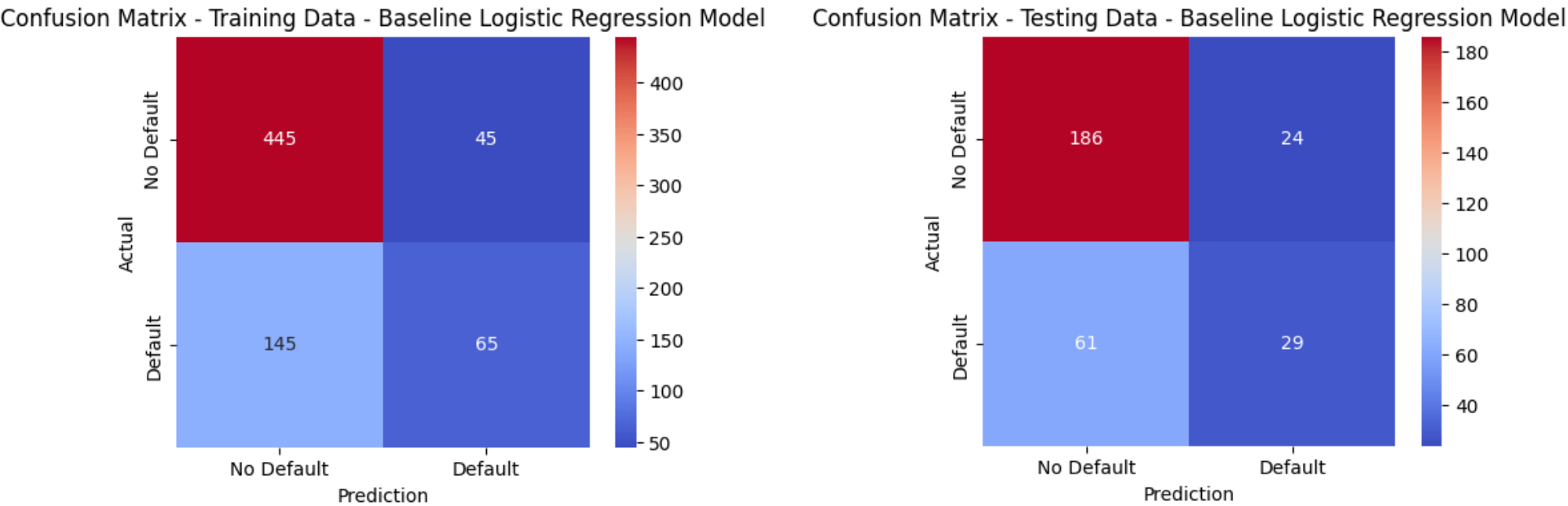
Feature Name	-	-	-	-	-
Checking Balance	Unknown	< 0 DM	1 - 200 DM	> 200 DM	-
Credit History	Critical	Poor	Good	Very Good	Perfect
Saving Balance	Unknown	< 100 DM	100 - 500 DM	500 - 1000 DM	> 1000 DM
Employment Duration	Unemployed	< 1 year	1 - 4 years	4 - 7 years	> 7 years
Housing	Others	Rent	Own	-	-
Job	Unemployed	Unskilled	Skilled	Management	-
Other Credits	None	Store	Bank	-	-

Analyzed categorical variables and encoded them using Ordinal encoding (Since they represent ordinal values).

- Identified Categorical and Continuous features and Standardize them using the appropriate method i.e. Min Max Scaling for Categorical and Standard Scaling for Continuous.
- Split the data between Train/Test split using a 70/30 ratio, use this ratio for all the models.
- Perform Standardization on 1 copy of Train/Test split to use it for Models like Logistic Regression as it requires Standardization. For Tree Based Models I used the Non Scaled Data.
- Checked for class imbalance and implemented synthetic data generation for nominal and categorical data using SMOTENC on Splitted data (Standardized and Non-standardized separately).

Model Development and Training

- Tested Logistic Regression Models (Base, With Class Weight = Balanced and Custom Threshold Boundaries)



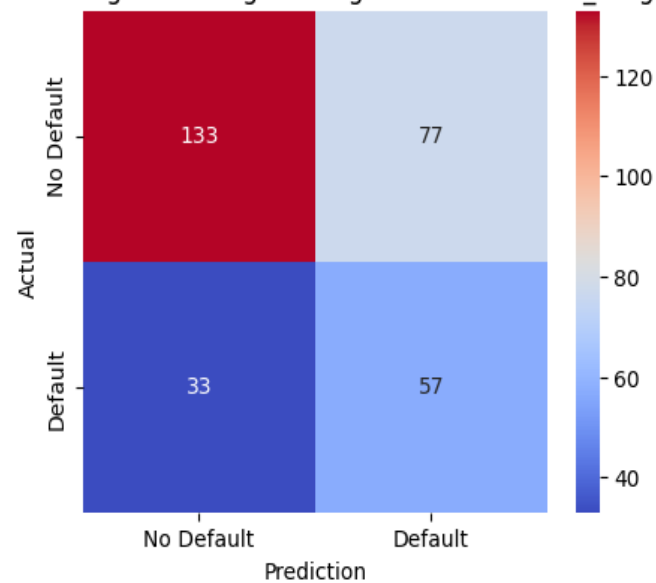
Logistic Regression Model Performance on Training and Test

Metric	Training Data	Test Data	Observations
Accuracy	72.85%	71.67%	Very close – no major overfitting
Precision (Class 1)	59.09%	54.71%	Slight drop, but expected
Recall (Class 1)	30.95%	32.22%	Slight improvement, but still low
F1-Score (Class 1)	40.62%	40.56%	Almost identical, indicating stable performance
Confusion Matrix Differences	TP: 65, FP: 45, FN: 145, TN: 445	TP: 29, FP: 24, FN: 61, TN: 186	Performance drops on test set, which is expected

Selected baseline models such as Logistic Regression to establish performance benchmarks.

- **Attempt to improve Logistic Regression using Class Balancing.**
Class_weights = ‘balanced’

Confusion Matrix - Testing Data - Logistic Regression with class_weight = balanced



Comparison of Baseline vs. Balanced Logistic Regression Model (Test Data)			
Metric	Baseline Model (Test Data)	Balanced Model (Test Data)	Observations
Accuracy	71.67%	63.33%	Accuracy dropped, but recall improved significantly.
Precision (Class 1)	54.71%	42.54%	More false positives, leading to lower precision.
Recall (Class 1)	32.22%	63.33%	Recall almost doubled, fewer missed defaulters.
F1-Score (Class 1)	40.56%	50.89%	Improved, better balance between precision and recall.
Confusion Matrix Differences	TP: 29, FP: 24, FN: 61, TN: 186	TP: 57, FP: 77, FN: 33, TN: 133	Recall improved, but false positives increased.

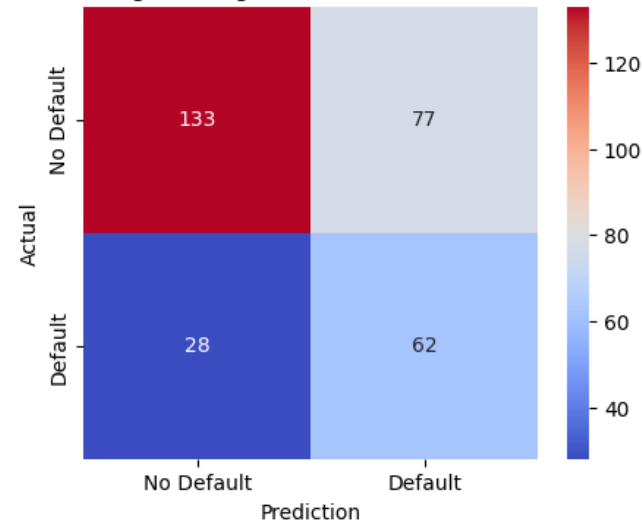
- Try decreasing the decision threshold from 0.5 to 0.3. Best Threshold was identified as 0.45.

Logistic Regression Decision Threshold Analysis						
Threshold	Accuracy	Precision (Class 1)	Recall (Class 1)	F1-Score	Confusion Matrix (TN, FP, FN, TP)	Observations
0.30	50.00%	36.49%	90.00%	51.92%	[[69 141] [9 81]]	Very high recall but way too many false positives (141 FP).
0.35	54.33%	38.07%	83.33%	52.26%	[[88 122] [15 75]]	Slightly better precision, recall still high, FP still very high.
0.40	56.67%	38.64%	75.56%	51.13%	[[102 108] [22 68]]	Recall drops, but still decent; FP reduced slightly.
0.45	60.67%	41.03%	71.11%	52.03%	[[118 92] [26 64]]	Best balance so far; recall still good, fewer FP.
0.50 (Baseline)	63.33%	42.54%	63.33%	50.89%	[[133 77] [33 57]]	Our baseline balanced model.
0.55	65.67%	44.25%	55.56%	49.26%	[[147 63] [40 50]]	Precision improves, but recall drops more.
0.60	68.33%	47.25%	47.78%	47.51%	[[162 48] [47 43]]	Balanced but recall is low.
0.65	70.67%	51.39%	41.11%	45.68%	[[175 35] [53 37]]	Highest precision, but recall is too low (bad for default detection).

Comparison of Baseline vs. Adjusted Logistic Regression Model (Test Data)			
Metric	Baseline Model (Threshold = 0.50)	Adjusted Model (Threshold = 0.45)	Observations
Accuracy	63.33%	60.67%	Slight drop in accuracy, expected due to recall improvement.
Precision (Class 1)	42.54%	41.03%	Small drop, more false positives introduced.
Recall (Class 1)	63.33%	71.11%	Recall improved, detecting more defaulters.
F1-Score (Class 1)	50.89%	52.03%	Improved balance between precision and recall.
Confusion Matrix Differences	TP: 57, FP: 77, FN: 33, TN: 133	TP: 64, FP: 92, FN: 26, TN: 118	More defaulters detected, but with increased false positives.

- Used SMOTENC to increase the performance further:

Confusion Matrix - Logistic Regression trained on SMOTE-balanced Data

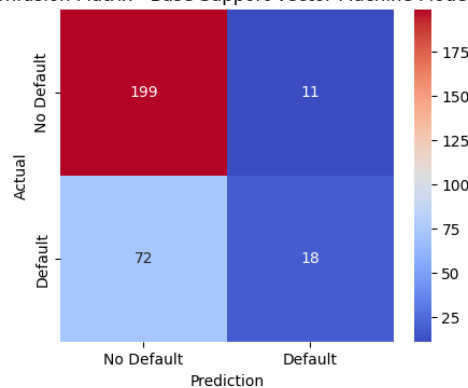


Comparison of Logistic Regression Models (Baseline vs. Adjusted Threshold vs. SMOTE)

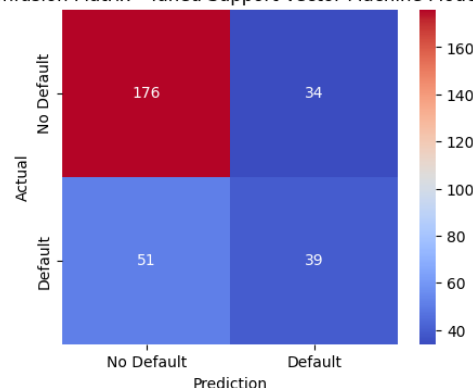
Metric	Baseline Model (Threshold = 0.50)	Adjusted Model (Threshold = 0.45)	SMOTE-balanced Model	Observations
Accuracy	63.33%	60.67%	65.00%	Slight improvement with SMOTE, recall remains high.
Precision (Class 1)	42.54%	41.03%	44.60%	SMOTE improved precision slightly.
Recall (Class 1)	63.33%	71.11%	68.89%	SMOTE maintains high recall, close to adjusted model.
F1-Score (Class 1)	50.89%	52.03%	54.15%	Best balance between precision and recall so far.
Confusion Matrix Differences	TP: 57, FP: 77, FN: 33, TN: 133	TP: 64, FP: 92, FN: 26, TN: 118	TP: 62, FP: 77, FN: 28, TN: 133	SMOTE increased TP while keeping FP the same.

Implemented Support Vector Machine (SVM) and Tuned It.

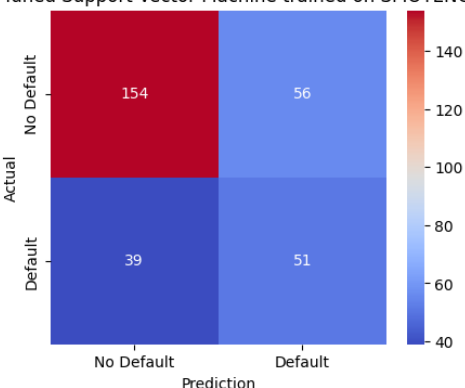
Confusion Matrix - Base Support Vector Machine Model



Confusion Matrix - Tuned Support Vector Machine Model



Confusion Matrix - Tuned Support Vector Machine trained on SMOTENC balanced data



Comparison of Base, Tuned, and Tuned SVM with SMOTENC

Metric	Base SVM	Tuned SVM	Tuned SVM + SMOTENC	Observations
Best Parameters	Default	C=10, gamma='scale', kernel='rbf'	C=10, gamma='scale', kernel='rbf'	Tuned SVM applied best parameters, SMOTENC balances the data.
Accuracy	72.33%	71.67%	68.33%	Base SVM had the highest accuracy, but recall was poor.
Precision (Class 1)	62.07%	53.42%	47.66%	Base SVM had the highest precision, meaning fewer false positives.
Recall (Class 1)	20.00%	43.33%	56.67%	SMOTENC improved recall significantly, meaning it detected more actual defaulters.
F1-Score (Class 1)	30.25%	47.85%	51.78%	SMOTENC provided the best balance between precision and recall.
ROC-AUC	69.30%	70.89%	71.14%	SMOTENC improved model separability between classes.
Confusion Matrix Differences	TP: 18, FP: 11, FN: 72, TN: 199	TP: 39, FP: 34, FN: 51, TN: 176	TP: 51, FP: 56, FN: 39, TN: 154	SMOTENC detected more defaults (TP) but increased false positives (FP).

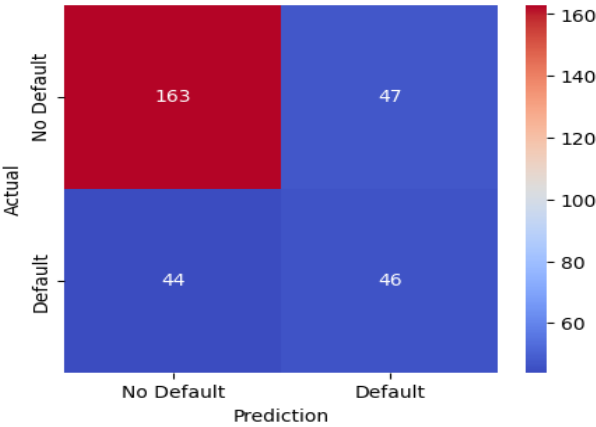
Experimented with Tree-Based Models:

Use RandomStates = 42 for all models.

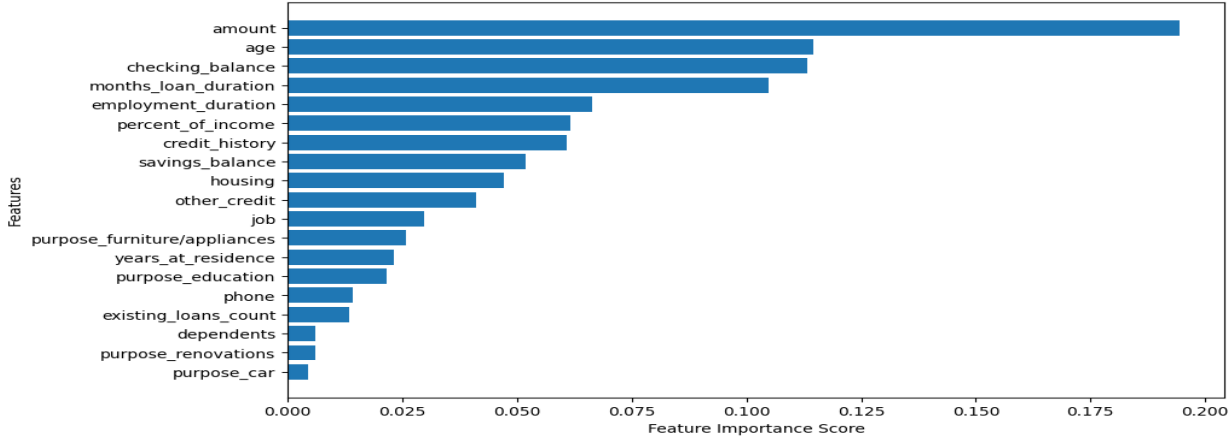
Train the base models and evaluate them

- Decision Tree
- Random Forest
- XGBoost
- LightGBM

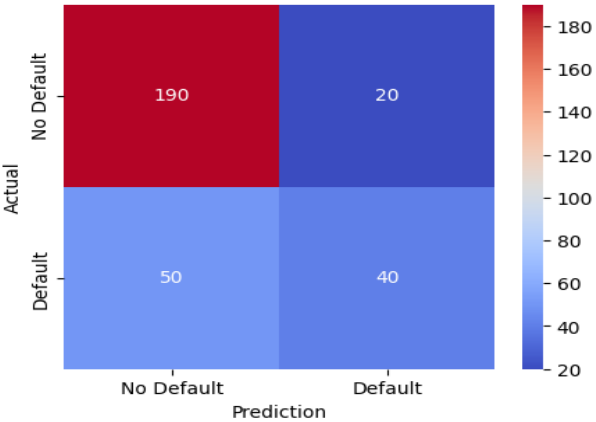
Confusion Matrix - DecisionTree on Base Data



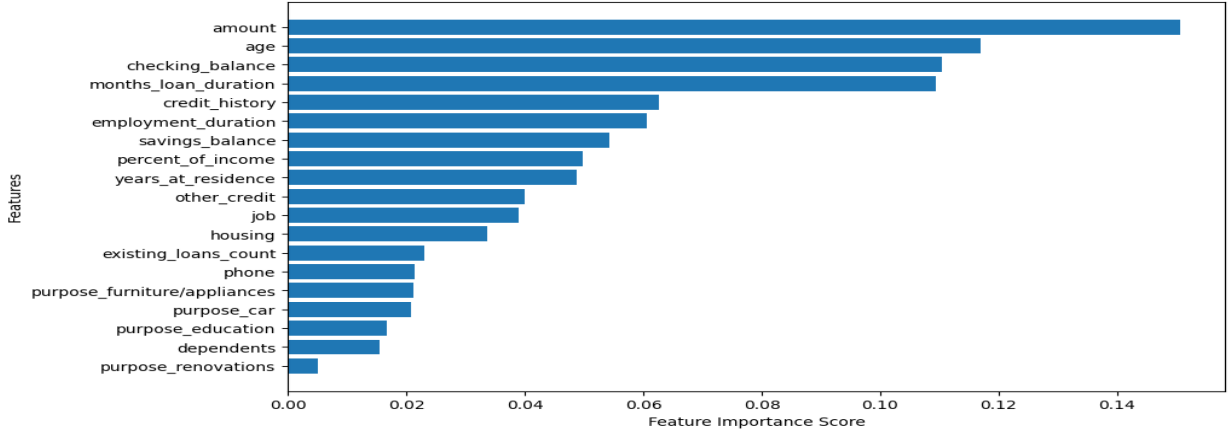
DecisionTree Feature Importance

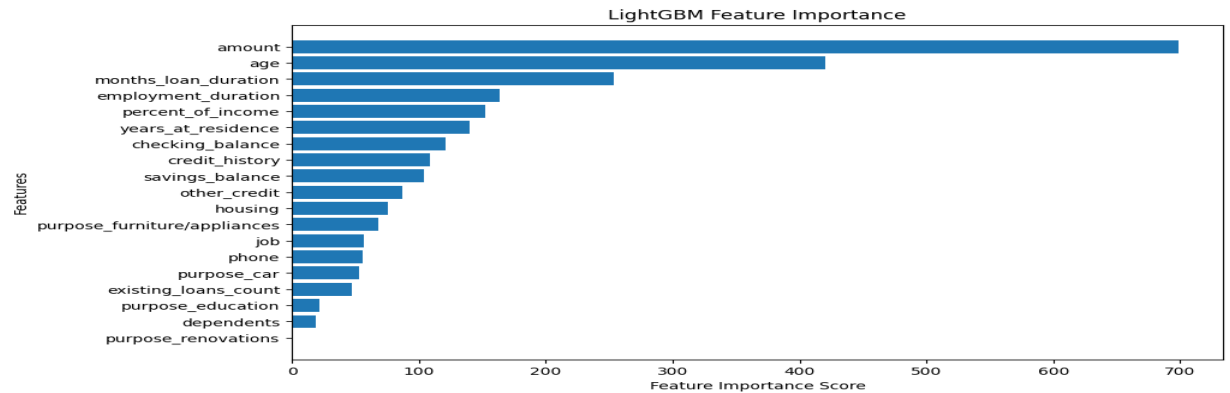
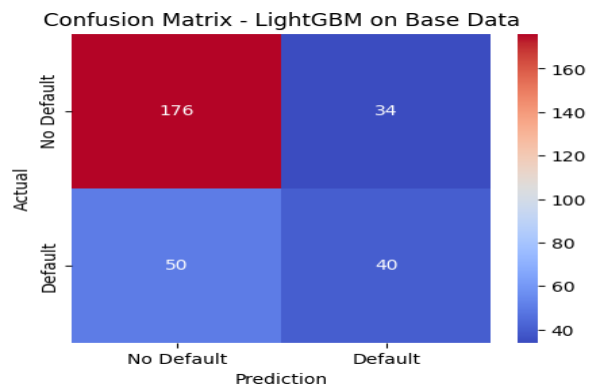
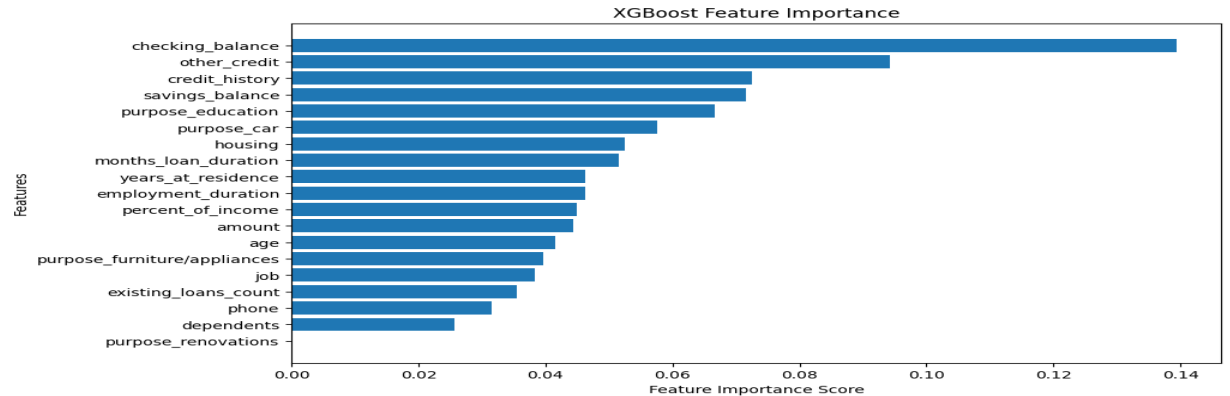
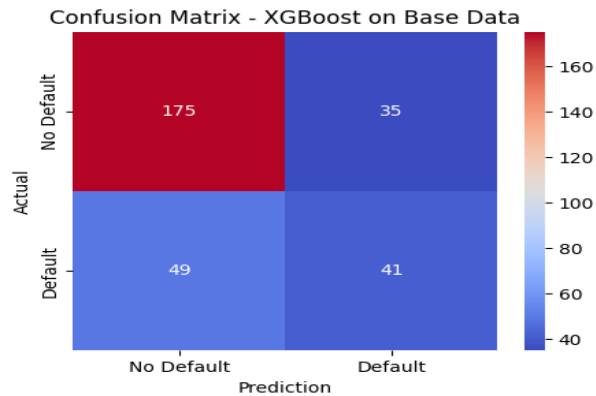


Confusion Matrix - RandomForest on Base Data



RandomForest Feature Importance

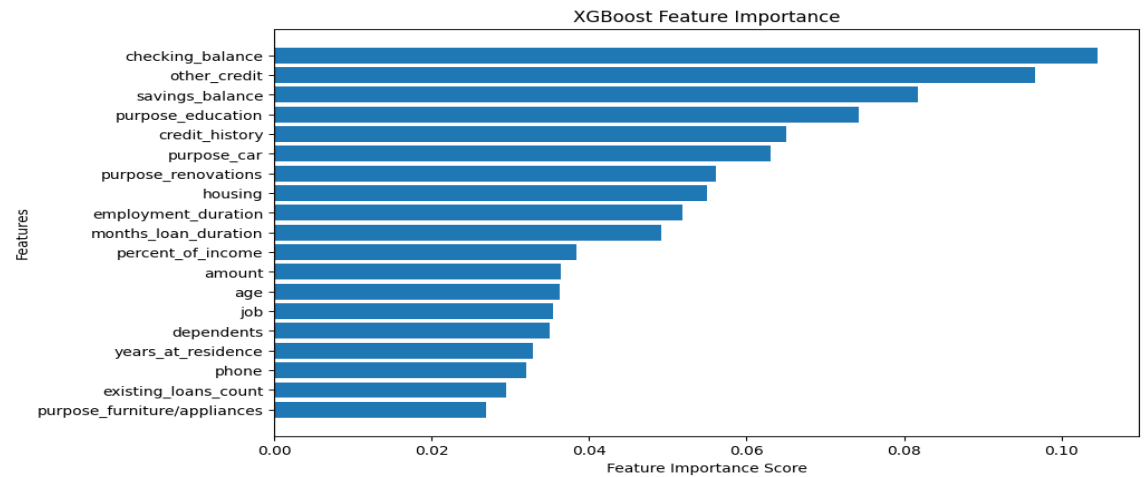
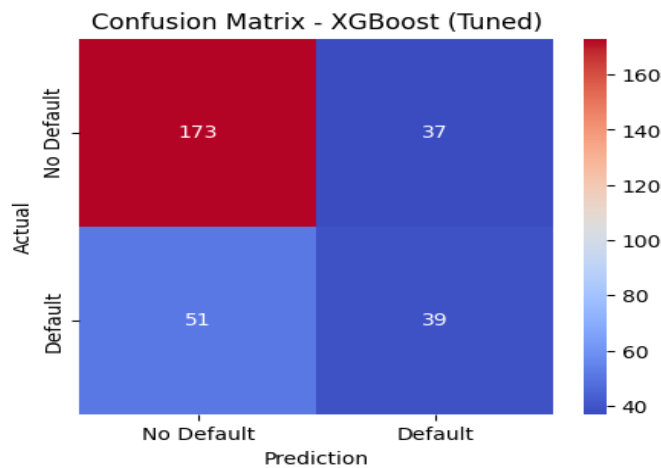
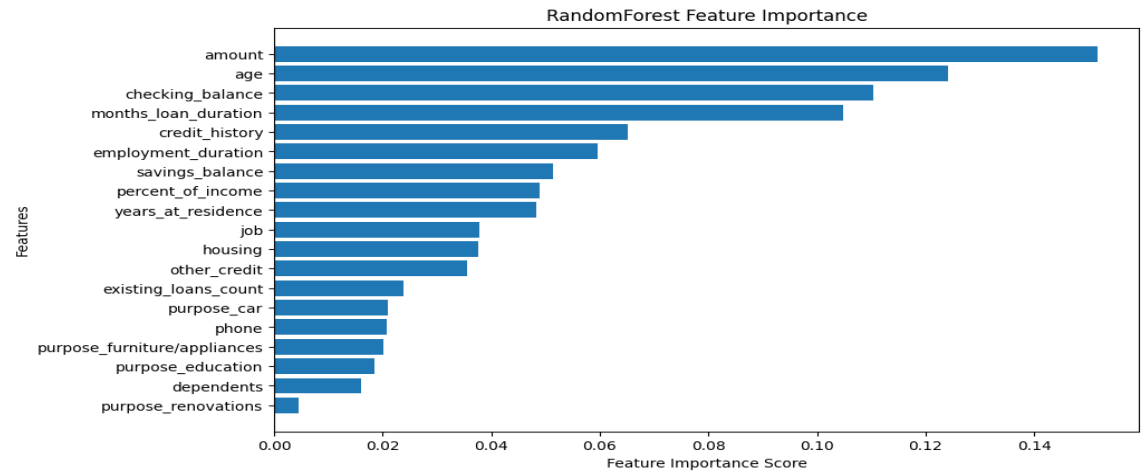
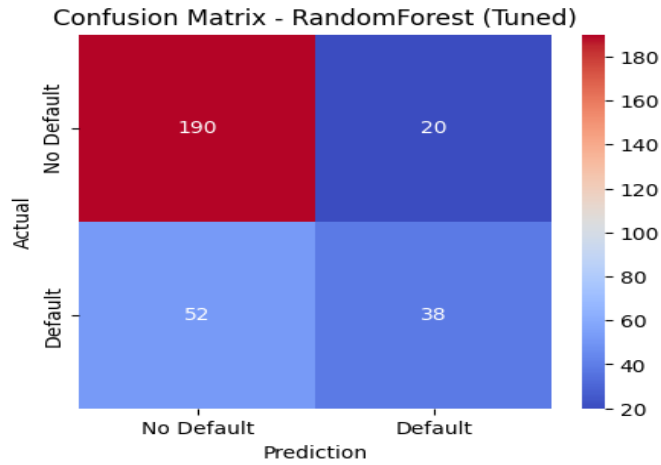


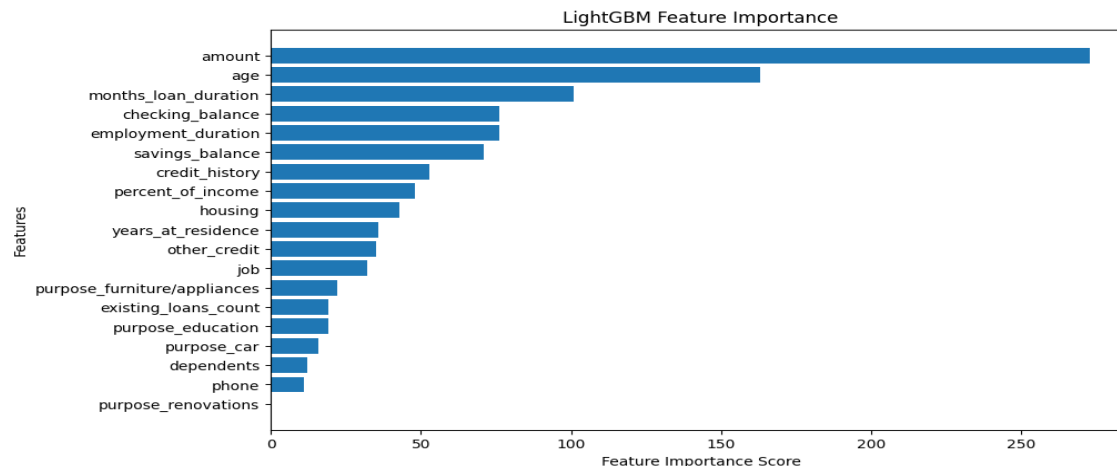
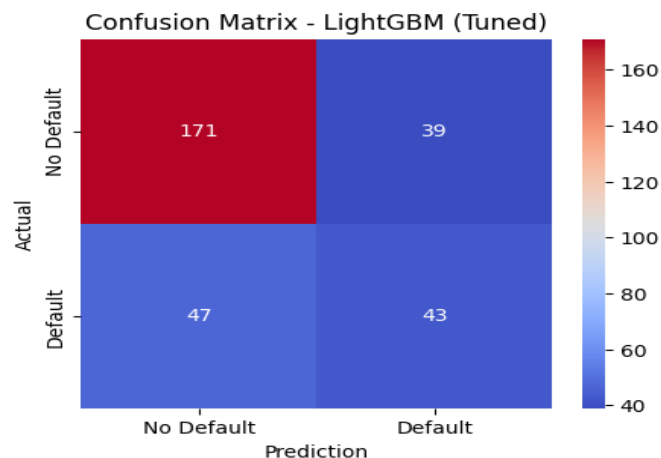


Comparison of Tree-Based Models on Base Data

Metric	Decision Tree	Random Forest	XGBoost	LightGBM	Observations
Accuracy	69.67%	76.67%	72.00%	72.00%	Random Forest had the highest accuracy.
Precision (Class 1)	49.46%	66.67%	53.95%	54.05%	Random Forest had the best precision, meaning fewer false positives.
Recall (Class 1)	51.11%	44.44%	45.56%	44.44%	Decision Tree had the best recall, meaning more defaulters were detected.
F1-Score (Class 1)	50.27%	53.33%	49.40%	48.78%	Random Forest achieved the best balance between precision and recall.
Confusion Matrix Differences	TP: 46, FP: 47, FN: 44, TN: 163	TP: 40, FP: 20, FN: 50, TN: 190	TP: 41, FP: 35, FN: 49, TN: 175	TP: 40, FP: 34, FN: 50, TN: 176	Decision Tree had the best recall but also the highest false positives.
Top 5 Important Features	Amount, Age, Checking Balance, Employment Duration, Percent of Income	Amount, Age, Checking Balance, Months Loan Duration, Credit History	Checking Balance, Other Credit, Credit History, Savings Balance, Purpose Education	Amount, Age, Months Loan Duration, Employment Duration, Percent of Income	Different models focus on different risk factors.

Perform Hyperparameter Tuning using GridSearchCV:



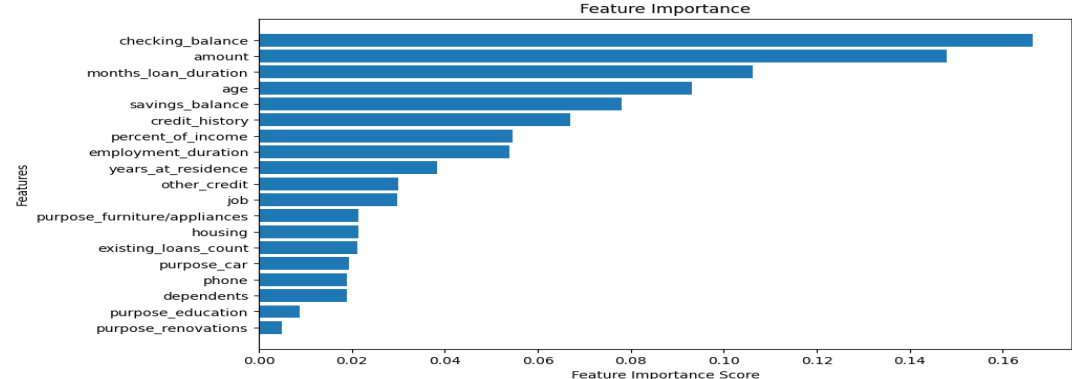
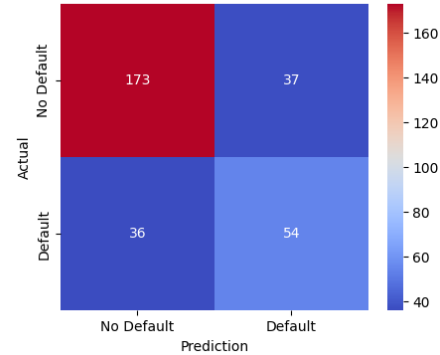


Comparison of Tuned Random Forest, XGBoost, and LightGBM on Base Data

Metric	Random Forest (Tuned)	XGBoost (Tuned)	LightGBM (Tuned)	Observations
Best Parameters	<code>max_depth=20,</code> <code>min_samples_leaf=1,</code> <code>min_samples_split=2,</code> <code>n_estimators=50</code>	<code>learning_rate=0.3,</code> <code>max_depth=9, n_estimators=200,</code> <code>subsample=0.8</code>	<code>learning_rate=0.1,</code> <code>max_depth=3,</code> <code>n_estimators=200,</code> <code>num_leaves=20</code>	XGBoost and LightGBM use boosting techniques, whereas Random Forest relies on bagging.
Accuracy	76.00%	70.67%	71.33%	Random Forest had the highest accuracy.
Precision(Class 1)	65.52%	51.32%	52.44%	Random Forest had the best precision.
Recall(Class 1)	42.22%	43.33%	47.78%	LightGBM detected the most defaulters.
F1-Score(Class 1)	51.35%	46.99%	50.00%	Random Forest still maintains the best balance.
Confusion Matrix Differences	TP: 38, FP: 20, FN: 52, TN: 190	TP: 39, FP: 37, FN: 51, TN: 173	TP: 43, FP: 39, FN: 47, TN: 171	LightGBM captured more defaulters but with more false positives.
Top 5 Important Features	Amount, Age, Checking Balance, Months Loan Duration, Credit History	Checking Balance, Other Credit, Saving Balance, Purpose Education, Credit History	Amount, Age, Months Loan Duration, Checking Balance, Employment Duration	Different models focus on different risk factors.

Used SMOTENC Balanced Data to Train the Tuned Models to Seek Further improvements:

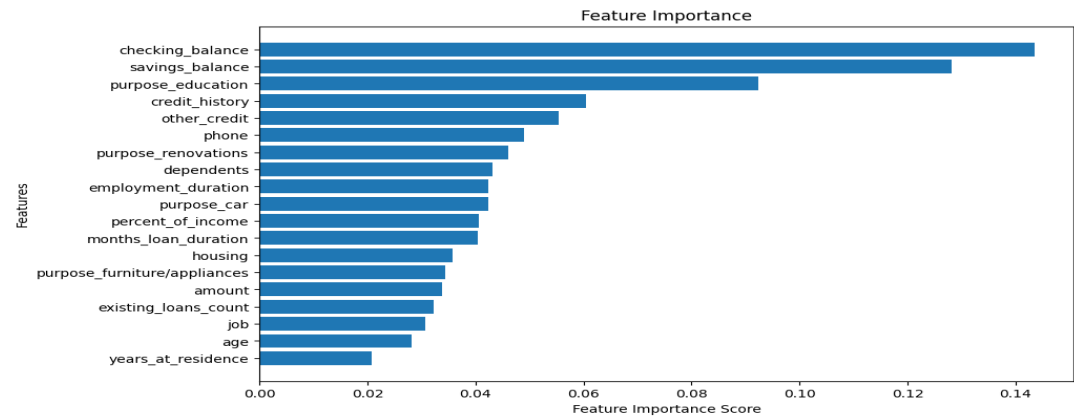
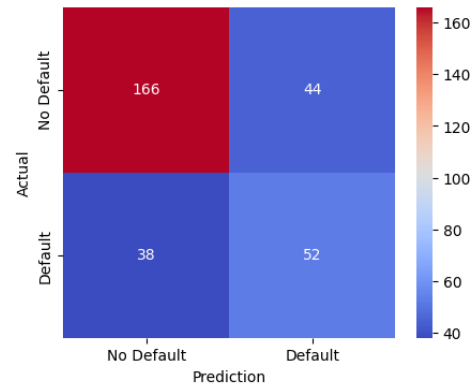
Confusion Matrix - Tunned RandomForrest trained on SMOTEENC balanced unscaled data



Comparison of Tuned Random Forest vs. Tuned + SMOTENC Random Forest

Metric	Random Forest (Tuned)	Random Forest (Tuned + SMOTENC)	Observations
Accuracy	76.00%	75.67%	Both models have similar accuracy.
Precision (Class 1 - Defaulters)	65.52%	59.34%	Tuned RF has better precision, meaning fewer false positives.
Recall (Class 1 - Defaulters)	42.22%	60.00%	SMOTENC RF has much better recall, detecting more actual defaulters.
F1-Score (Class 1)	51.35%	59.67%	SMOTENC RF balances precision and recall better.
Confusion Matrix Differences	TP: 38, FP: 20, FN: 52, TN: 190	TP: 54, FP: 37, FN: 36, TN: 173	SMOTENC RF detects more defaulters but at the cost of more false positives.
Top 5 Important Features	Amount, Age, Checking Balance, Months Loan Duration, Credit History	Checking Balance, Amount, Months Loan Duration, Age, Savigns Balance	Both model gives different important to features.

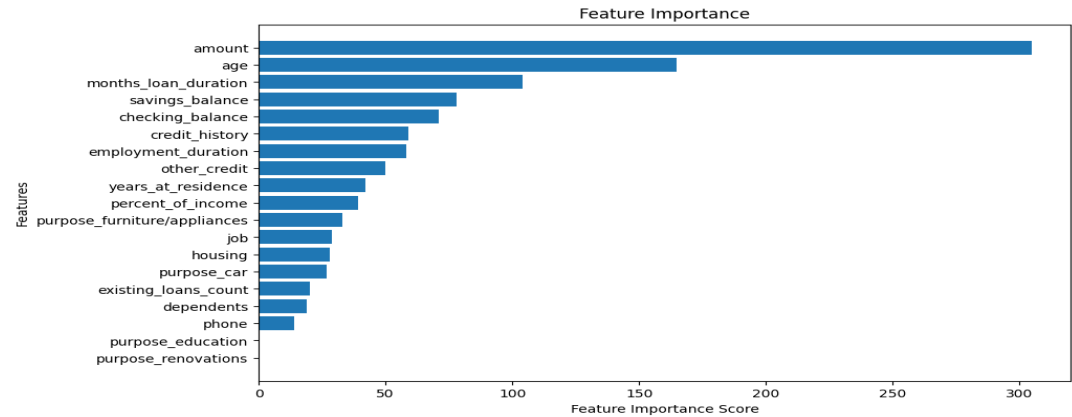
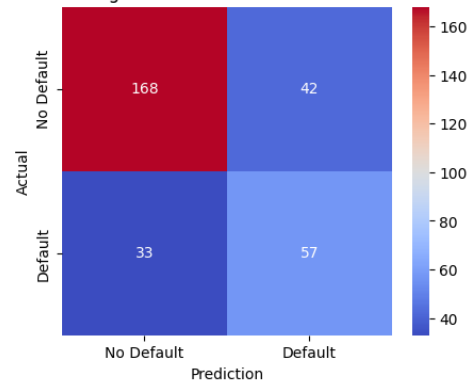
Confusion Matrix - Tunned XGBoost trained on SMOTEENC balanced unscaled data



Comparison of Tuned XGBoost vs. Tuned + SMOTENC XGBoost

Metric	XGBoost (Tuned)	XGBoost (Tuned + SMOTENC)	Observations
Accuracy	70.67%	72.67%	SMOTENC XGBoost slightly improved accuracy.
Precision (Class 1 - Defaulters)	51.32%	54.17%	SMOTENC improved precision slightly.
Recall (Class 1 - Defaulters)	43.33%	57.78%	SMOTENC significantly improved recall, detecting more actual defaulters.
F1-Score (Class 1)	46.99%	55.91%	SMOTENC XGBoost had a better balance between precision and recall.
Confusion Matrix Differences	TP: 39, FP: 37, FN: 51, TN: 173	TP: 52, FP: 44, FN: 38, TN: 166	SMOTENC XGBoost captured more defaulters but had slightly more false positives.
Top 5 Important Features	Checking Balance, Other Credit, Saving Balance, Purpose Education, Credit History	Checking Balance, Savings Balance, Purpose Education, Credit History, Other Credits	Both model gives different important to features.

Confusion Matrix - Tunned LightGBM trained on SMOTEENC balanced unscaled data



Comparison of Tuned LightGBM vs. Tuned LightGBM with SMOTENC on Base Data

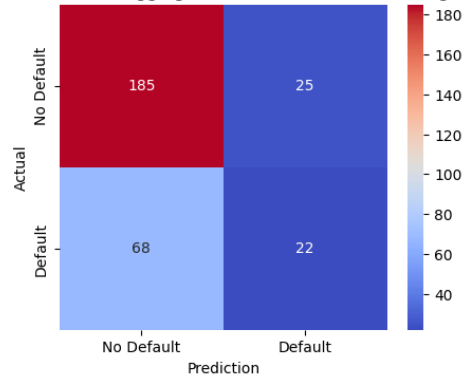
Metric	LightGBM (Tuned)	LightGBM (Tuned & SMOTENC)	Observations
Best Parameters	learning_rate=0.1, max_depth=3, n_estimators=200, num_leaves=20	learning_rate=0.1, max_depth=3, n_estimators=200, num_leaves=20	Same best parameters were used for both models.
Accuracy	71.33%	75.00%	LightGBM with SMOTENC improved accuracy.
Precision (Class 1)	52.44%	57.58%	LightGBM with SMOTENC had better precision, reducing false positives.
Recall (Class 1)	47.78%	63.33%	LightGBM with SMOTENC detected more defaulters.
F1-Score (Class 1)	50.00%	60.32%	LightGBM with SMOTENC had a better balance of precision and recall.
Confusion Matrix Differences	TP: 43, FP: 39, FN: 47, TN: 171	TP: 57, FP: 42, FN: 33, TN: 168	SMOTENC improved recall but slightly increased false positives.
Top 5 Important Features	Amount, Age, Months Loan Duration, Checking Balance, Employment Duration	Amount, Age, Months Loan Duration, Savings Balance, Checking Balance	Both model gives different important to features.

Experiment with Ensemble Methods:
Use RandomState = 42 for all models.

- Bagging
- Boosting

Implement Bagging on KNN as Estimator

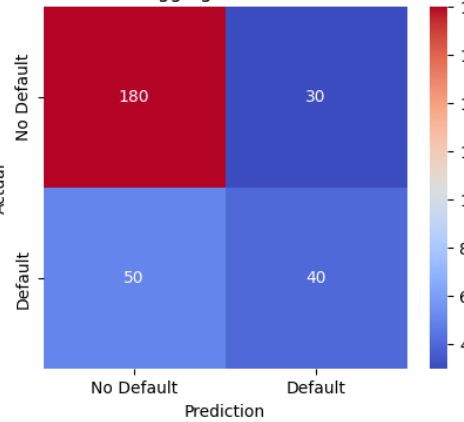
Confusion Matrix - Bagging Classifier with K-Nearest Neighbors



Comparison of Bagging Classifier with K-Nearest Neighbors and Tuned Random Forest			
Metric	Bagging with KNN	Random Forest (Tuned)	Observations
Accuracy	69.00%	76.00%	Random Forest performed better in overall accuracy.
Precision (Class 1)	46.81%	65.52%	Random Forest had significantly better precision.
Recall (Class 1)	24.44%	42.22%	Random Forest detected more defaulters.
F1-Score (Class 1)	32.12%	51.35%	Random Forest had a better balance between precision and recall.
Confusion Matrix Differences	TP: 22, FP: 25, FN: 68, TN: 185	TP: 38, FP: 20, FN: 52, TN: 190	Bagging KNN struggled with recall, missing a large number of defaulters.

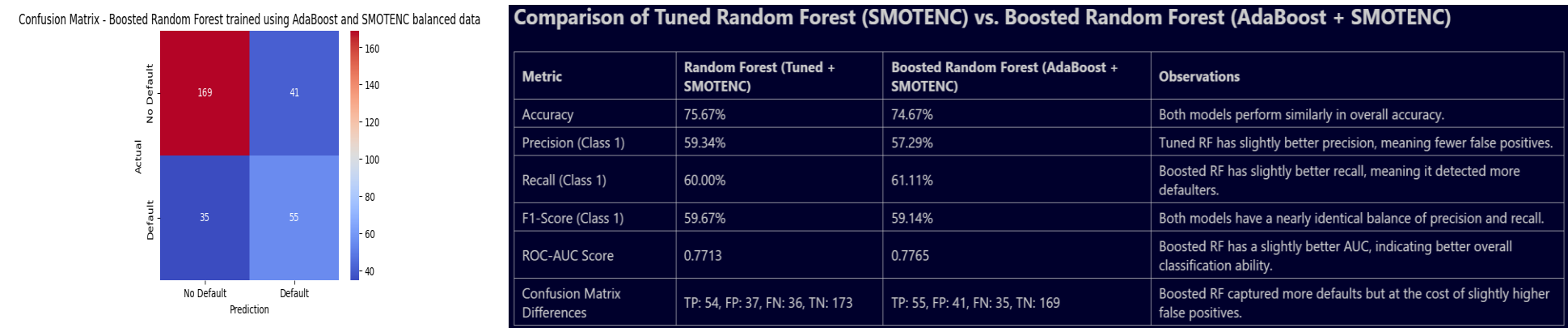
Implement Bagging on DecisionTree as Estimator

Confusion Matrix - Bagging Classifier with Decision Trees



Comparison of Bagging Classifier with Decision Trees and Tuned Random Forest			
Metric	Bagging with Decision Trees	Random Forest (Tuned)	Observations
Accuracy	73.33%	76.00%	Random Forest performed slightly better.
Precision (Class 1)	57.14%	65.52%	Random Forest had better precision.
Recall (Class 1)	44.44%	42.22%	Bagging with Decision Trees had slightly better recall.
F1-Score (Class 1)	50.00%	51.35%	Random Forest maintained a better balance.
Confusion Matrix Differences	TP: 40, FP: 30, FN: 50, TN: 180	TP: 38, FP: 20, FN: 52, TN: 190	Bagging Decision Trees had higher false positives.

Implement AdaBoost Using RandomForest as an Estimator trained on Balanced Data.



All the **Models** were evaluated using a **Common Evaluation and Performance Metrics**

- Evaluated models based on Accuracy, Precision, Recall, F1-score, and ROC-AUC.
- Plotted confusion matrices to analyze false positives and false negatives.
- Generated classification reports to summarize performance across different models.
- Analyzed feature importance for tree-based models to understand the impact of individual predictors.

Model Selection and Business Insights

- Compared performance across all models and selected the best-performing one based on mainly Recall, F1-Score and ROC-AUC.
- Analyzed key features influencing loan default prediction to derive actionable business insights.
- Recommended potential improvements for future research, such as incorporating external data sources (e.g., credit scores, employment history).
- Documented findings, insights, and limitations for final reporting.

This structured approach ensures transparency, reproducibility, and interpretability of the loan default prediction study.

Results

The performance of different models was assessed based on multiple evaluation metrics. Among all the tested models, LightGBM emerged as the best-performing model when trained on SMOTENC-balanced data. The key results are as follows:

- **LightGBM Model (Best Performing Model)**
 - Accuracy: 75.00%
 - Precision: 57.58%
 - Recall: 63.33%
 - F1-Score: 60.32%
 - ROC-AUC: 0.7837
 - Top 10 Important Features: Loan Amount, Age, Loan Duration, Savings Balance, Checking Balance, Credit History, Employment Duration, Other Credits, Years At Residence, Percent of Income.

Other models, including Logistic Regression and Random Forest, showed moderate performance but were outperformed by boosting techniques. The impact of data balancing was evident in improving recall, indicating a higher ability to correctly identify defaulters.

Discussion

The results highlight that boosting-based models, particularly LightGBM, are highly effective in predicting loan defaults. The use of SMOTENC balancing improved recall, ensuring that more actual defaulters were correctly classified. This is particularly important in financial applications, where missing a defaulter can result in significant losses. However, the precision of the model suggests some level of false positives, indicating that non-defaulters were misclassified as defaulters.

The study also reveals that financial attributes such as loan amount, savings balance, and checking balance are strong indicators of loan default risk. These insights align with previous financial risk research, where liquidity and credit history play a crucial role in assessing loan repayment capacity.

A limitation of the study is the potential bias introduced by the dataset. The model's performance could vary when applied to real-world data with different distributions. Future research could focus on incorporating additional financial variables, external credit scores, and behavioral analytics to enhance model robustness. Furthermore, testing deep learning approaches could provide insights into non-linear relationships within the dataset.

Conclusion

This study demonstrates the effectiveness of machine learning techniques in loan default prediction. LightGBM, trained on balanced data, provided the best performance, achieving a recall of 63.33% and an AUC score of 0.7837. These findings suggest that financial institutions can leverage machine learning to optimize risk assessment and lending decisions. Future work should focus on refining feature selection, incorporating external data sources, and exploring more advanced predictive techniques to enhance accuracy and precision.